

07 — Backend Developer

[← Powrót do README](#) (../README.md) | [Następna: Frontend Developer](#) → (./frontend-developer.md)

Zakres odpowiedzialności

Jako Backend Developer w projekcie IOC Service jesteś odpowiedzialny za:

- Implementację Service Layer i API endpoints
 - Architekturę integracji (Adapter Pattern) — **kluczowy obszar**
 - Modele danych i migracje (SQLAlchemy + Alembic)
 - Background Worker i scheduling
 - Performance optimization i caching
 - Unit i integration tests
-

Instrukcje per Milestone

M1.4.2 — Security (Twój udział: ~60%)

Zadania:

1. **Model User** — SQLAlchemy model z polami: id, username, password_hash, role, is_active, created_at, last_login
2. **Auth endpoints** — /auth/login (POST), /auth/logout (POST), /auth/me (GET)
3. **Password hashing** — argon2id (argon2-cffi library)
4. **Session management** — Redis-backed Flask session
5. **RBAC middleware** — @require_permission decorator
6. **CSRF middleware** — Token generation/validation w before_request
7. **JWT endpoint** — /auth/token (POST) dla API clients

Przykład: Model User

```
# app/models.py – dodaj model User
from argon2 import PasswordHasher
from sqlalchemy import Column, Integer, String, Boolean, DateTime
from app.db import Base

ph = PasswordHasher(
    time_cost=3,
    memory_cost=65536,
    parallelism=4,
    hash_len=32,
    salt_len=16,
)

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    username = Column(String(100), unique=True, nullable=False, index=True)
    password_hash = Column(String(256), nullable=False)
    role = Column(String(20), nullable=False, default="viewer")
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, server_default=func.now())
    last_login = Column(DateTime, nullable=True)

    def set_password(self, password: str) -> None:
        if len(password) < 12:
            raise ValueError("Password must be at least 12 characters")
        self.password_hash = ph.hash(password)

    def check_password(self, password: str) -> bool:
        try:
            return ph.verify(self.password_hash, password)
        except Exception:
            return False
```

M1.5.0 — Modularization (Twój udział: ~80%)

Zadania:

1. **Split main.py** — Extract app factory, move business logic to services
2. **Service Layer** — IndicatorService, FeedService, ExportService
3. **Split routes/ops.py** — admin.py, sync_jobs.py, settings.py

Przykład: IndicatorService

```

# app/services/indicator_service.py
from typing import List, Optional, Dict, Any
from sqlalchemy.orm import Session
from app.models import Indicator
from app.query_parser import parse_query

class IndicatorService:
    """Business logic for indicators – single responsibility."""

    def __init__(self, db: Session):
        self.db = db

    def search(
        self,
        query: Optional[str] = None,
        source: Optional[str] = None,
        ioc_type: Optional[str] = None,
        is_active: Optional[bool] = True,
        limit: int = 100,
        offset: int = 0,
    ) -> tuple[List[Indicator], int]:
        """Search indicators z filtrowaniem i paginacją."""
        q = self.db.query(Indicator)

        if query:
            filters = parse_query(query)
            q = q.filter(*filters)
        if source:
            q = q.filter(Indicator.source == source)
        if ioc_type:
            q = q.filter(Indicator.type == ioc_type)
        if is_active is not None:
            q = q.filter(Indicator.is_active == is_active)

        total = q.count()
        items = q.order_by(Indicator.last_seen.desc()).offset(offset).limit(limit).all()

        return items, total

    def get_by_value(self, value: str) -> Optional[Indicator]:
        return self.db.query(Indicator).filter(Indicator.value == value).first()

    def get_stats(self) -> Dict[str, Any]:
        """Aggregate statistics."""
        from sqlalchemy import func

        return {
            "total": self.db.query(func.count(Indicator.id)).scalar(),
            "active": self.db.query(func.count(Indicator.id).filter(
                Indicator.is_active == True
            ).scalar(),
            "by_source": dict(
                self.db.query(
                    Indicator.source, func.count(Indicator.id)
                ).group_by(Indicator.source).all()
            ),
            "by_type": dict(
                self.db.query(
                    Indicator.type, func.count(Indicator.id)
                ).group_by(Indicator.type).all()
            )
        }

```

```
    },
}
```

M1.6.1 — Integration Adapters 🎯 (Twój udział: ~90%)

To jest **KLUCZOWY** milestone. Szczegóły: [03-integration-architecture.md](#) (../03-integration-architecture.md)

Twoje zadania:

1. Implementuj `contracts.py` — Protocol, DTOs, Value Objects
2. Implementuj `base.py` — BaseFeedAdapter
3. Implementuj `registry.py` — FeedAdapterRegistry
4. Implementuj `pipeline/ingestion.py` — IngestionPipeline
5. Migruj 5 connectorów na adapter pattern
6. Worker v2 z APScheduler + registry-driven scheduling
7. Feed config YAML loader
8. Adapter template + developer documentation

API Design Guidelines

RESTful Conventions

GET	/api/v1/indicators	→ List indicators (search, filter, paginate)
GET	/api/v1/indicators/{id}	→ Get single indicator
GET	/api/v1/indicators/ export	→ Export indicators (format param)
GET	/api/v1/feeds	→ List configured feeds
GET	/api/v1/feeds/{source_id}	→ Get feed details + stats
POST	/api/v1/feeds/{source_id}/sync	→ Trigger manual sync
PUT	/api/v1/feeds/{source_id}	→ Update feed configuration
GET	/api/v1/health	→ Health check (liveness)
GET	/api/v1/ready	→ Readiness check
GET	/api/v1/metrics	→ Prometheus metrics

Error Response Format

```
# Standardowy format błędu
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid query parameter: source",
    "details": [
      {"field": "source", "issue": "Unknown source 'xyz'. Available: crowdsec,
misp, mwdb"}
    ],
    "request_id": "req_abc123",
    "timestamp": "2026-04-07T12:00:00Z"
  }
}

# HTTP Status Codes
200 OK           → Success
201 Created      → Resource created
400 Bad Request  → Validation error
401 Unauthorized → Authentication required
403 Forbidden    → Insufficient permissions
404 Not Found    → Resource not found
429 Too Many    → Rate limit exceeded
500 Internal     → Server error (log details, return generic message)
```

Pagination Pattern

```
# Request: GET /api/v1/indicators?page=2&per_page=50&sort=-last_seen

# Response:
{
  "data": [...],
  "pagination": {
    "page": 2,
    "per_page": 50,
    "total": 15432,
    "total_pages": 309,
    "has_next": true,
    "has_prev": true
  }
}
```

Database Guidelines

Migrations

```
# Tworzenie nowej migracji
alembic revision --autogenerate -m "add_users_table"

# Aplikowanie migracji
alembic upgrade head

# Rollback ostatniej migracji
alembic downgrade -1

# Sprawdzenie statusu
alembic current
alembic history
```

Zasady migracji

1. **NIGDY** nie edytuj istniejącej migracji po merge do main
2. **ZAWSZE** testuj downgrade (rollback)
3. **UNIKAJ** destructive operations (DROP COLUMN) — najpierw deprecate
4. **DODAWAJ** indeksy CONCURRENTLY w produkcji
5. **ROZDZIELAJ** schema changes od data migrations

Testing Requirements

Minimalne wymagania

Typ testu	Coverage target	Gdzie
Unit tests	>85%	Services, Adapters, Pipeline
Integration tests	>70%	API endpoints, DB operations
Contract tests	100%	Każdy FeedAdapter
Performance tests	Benchmarks	Export, Search, Ingestion

Fixtures

```
# tests/conftest.py – kluczowe fixtures

@pytest.fixture
def db_session():
    """In-memory SQLite session dla unit tests."""
    engine = create_engine("sqlite:///memory:")
    Base.metadata.create_all(engine)
    session = Session(engine)
    yield session
    session.close()

@pytest.fixture
def indicator_service(db_session):
    return IndicatorService(db=db_session)

@pytest.fixture
def fake_adapter():
    """Fake adapter dla testów pipeline."""
    class FakeAdapter(BaseFeedAdapter):
        @property
        def source_id(self): return "fake"
        @property
        def display_name(self): return "Fake Source"
        @property
        def capabilities(self): return FeedCapabilities(...)

        def _fetch_raw(self, config, context):
            return [
                {"ip": "1.2.3.4", "type": "ip", "confidence": 80},
                {"ip": "5.6.7.8", "type": "ip", "confidence": 60},
            ]

        def _map_to_canonical(self, raw, config):
            return CanonicalIO(
                value=raw["ip"],
                type=IOType.IP,
                source="fake",
                source_ref=f"fake:{raw['ip']}",
                confidence=raw["confidence"],
            )

    return FakeAdapter()
```

✓ Code Review Checklist

- [] Type hints na wszystkich public methods
- [] Docstrings (Google style) na klasach i public methods
- [] Error handling: nie łykaj wyjątków cicho
- [] Logging: structured (extra=dict), nie f-string messages
- [] SQL: parameterized queries, nigdy string concatenation
- [] Tests: min 1 happy path + 1 error path per function
- [] Imports: stdlib → third-party → local (isort)
- [] Constants: nie magic numbers, używaj named constants

- [] Security: input validation na endpoint level
- [] Performance: lazy loading, pagination dla list endpoints



Development Environment Setup

```
# 1. Clone i setup
git clone <repo-url> ioc-service
cd ioc-service
python -m venv venv
source venv/bin/activate
pip install -e ".[dev]"

# 2. Start dependencies
docker compose up -d postgres redis

# 3. Run migrations
alembic upgrade head

# 4. Create admin user
python -c "from app.cli import create_user; create_user('admin', 'changeme123!', 'admin')"

# 5. Run app
flask run --debug --port 8080

# 6. Run worker
python -m app.worker

# 7. Run tests
pytest -v --cov=app

# 8. Linting
black app/ tests/
ruff check app/ tests/
mypy app/
```